

Programación de servicios web

Profesor. Jose Navarro Rios

Actividad 3.2: Diseño de Servicios Web RESTful

Equipo 4.

Hernández Garcia Hector Gabriel

Mexicano Romero Gerardo Misael

Pardo Correa Gonzalo

Ramirez Torres Jesus Adrian



CENTRO UNIVERSITARIO DE TONALÁ

Introducción

En la actividad anterior concluimos que REST es la arquitectura idónea para el proyecto y proponemos de manera inicial cinco endpoints solo para el catálogo de productos. Este diseño extiende esa propuesta a todas las entidades del sistema.

El objetivo de este documento es especificar esa API: sus recursos, endpoints, métodos, parámetros, respuestas, errores y reglas de seguridad, de modo que cualquier cliente (web, móvil o punto de venta) pueda consumirla.

Descripción general de la API

Es una API RESTful sin estado que expone los módulos administrativos de Farmacia Vital sobre HTTP y JSON.

Característica	Definición
Arquitectura	REST, comunicación sin estado (stateless)
Tecnología	Laravel (PHP) con Laravel Sanctum
URL base	<code>https://{host}/api/v1</code>
Versionado	En la ruta (/v1)
Formato de intercambio	JSON (<code>application/json</code> , UTF-8)
Autenticación	Token Bearer emitido en el inicio de sesión
Autorización	Roles y permisos por recurso y acción
Alcance	14 recursos y 117 servicios
Convención de nombres	Recursos en plural y kebab-case; sustantivos, nunca verbos
Listados	Paginados, con búsqueda, orden y filtros

El estado actual del código (Laravel, MVC monolítico) condiciona tres decisiones:

- `routes/api.php` solo tiene GET `/user` con `auth:sanctum`; toda la API es nueva y se versiona en `/api/v1`.
- El CRUD completo con borrado lógico, restauración y borrado físico ya existe para productos en `ProductoController`, pero responde con vistas Blade. La API replica esa lógica devolviendo JSON.
- Solo la tabla `productos` tiene `deleted_at`. Las tablas de recetas, doctores, etc. necesitan una migración con `softDeletes()` para soportar eliminación lógica y restauración.

Recursos del sistema

Entidades del proyecto

El proyecto tiene 15 tablas propias: 14 entidades y una tabla pivote (`usuario_rol`).

Entidad (modelo)	Tabla	Llave primaria	Campos	Relaciones
Producto	<code>productos</code>	<code>id</code>	nombre, precio, stock, categoria, imagen, created_at, updated_at, deleted_at	Ninguna por ahora
Usuario	<code>usuarios</code>	<code>id_usuario</code>	nombre, apellido, email (único), password, telefono, activo, fecha_registro	Tiene roles, médico, citas, carritos, lista de deseos, transacciones, logs, sesiones sociales
Rol	<code>roles</code>	<code>id_rol</code>	nombre, descripcion	Pertenece a muchos usuarios
Especialidad	<code>especialidades</code>	<code>id_especialidad</code>	nombre, descripcion	Tiene muchos médicos
SesionSocial	<code>sesiones_sociales</code>	<code>id_sesion</code>	<code>id_usuario</code> , proveedor, provider_id, fecha_vinculacion	Pertenece a usuario
Medico	<code>medicos</code>	<code>id_medico</code>	<code>id_usuario</code> (único), cedula (única),	Pertenece a usuario y

Entidad (modelo)	Tabla	Llave primaria	Campos	Relaciones
			especialidad_id, consultorio	especialidad; tiene citas
Cita	citas	id_cita	paciente_id, medico_id, fecha, hora, estado, motivo	Pertenece a usuario (paciente) y médico; tiene un expediente
ExpedienteMedico	expediente_medico	id_expediente	id_cita (único), diagnostico, tratamiento, observaciones, fecha	Pertenece a cita
ServicioMedico	servicios_medicos	id_servicio	nombre, descripcion, costo, activo	Aparece en detalles de carrito y listas de deseos
Carrito	carrito	id_carrito	id_usuario, fecha, estado	Pertenece a usuario; tiene detalles y transacciones
CarritoDetalle	carrito_detalle	id_detalle	id_carrito, id_servicio, cantidad	Pertenece a carrito y servicio médico
ListaDeseo	lista_deseos	id_lista	id_usuario, id_servicio, fecha_agregado	Pertenece a usuario y servicio médico
HistorialTransaccion	historial_transacciones	id_transaccion	id_usuario, id_carrito, total, estado, fecha	Pertenece a usuario y carrito
Log	logs	id_log	id_usuario, accion, modulo, ip, fecha	Pertenece a usuario
UsuarioRol (pivote)	usuario_rols	id_usuario + id_rol	Solo las dos llaves	Une usuarios y roles

Las llaves foráneas con onDelete('restrict') son las que generan conflictos de integridad (409):
medicos.especialidad_id, citas.medico_id, carrito_detalle.id_servicio e
historial_transacciones.id_carrito.

Recursos publicados

Cada entidad se publica como un recurso independiente, con nombre en plural y kebab-case, igual que los módulos del panel /admin. La tabla pivote no es un recurso: se administra como subrecurso del usuario.

Módulo	Recurso	URI base	Identificador
Productos	Producto	/api/v1/productos	{id}
Usuarios	Usuario	/api/v1/usuarios	{id} = id_usuario
Roles	Rol	/api/v1/roles	{id} = id_rol
Especialidades	Especialidad	/api/v1/especialidades	{id} = id_especialidad
Sesiones sociales	Sesión social	/api/v1/sesiones-sociales	{id} = id_sesion
Médicos	Médico	/api/v1/medicos	{id} = id_medico
Citas	Cita	/api/v1/citas	{id} = id_cita
Expedientes	Expediente médico	/api/v1/expedientes	{id} = id_expediente
Servicios médicos	Servicio médico	/api/v1/servicios-medicos	{id} = id_servicio
Carritos	Carrito	/api/v1/carritos	{id} = id_carrito
Detalles de carrito	Detalle de carrito	/api/v1/carrito-detalles	{id} = id_detalle
Listas de deseos	Lista de deseos	/api/v1/listas-deseos	{id} = id_lista
Historial de transacciones	Transacción	/api/v1/historial-transacciones	{id} = id_transaccion
Logs	Log de auditoría	/api/v1/logs	{id} = id_log
Autenticación	Sesión administrativa	/api/v1/auth	No aplica

Servicios adicionales: PUT /api/v1/usuarios/{id}/roles (asigna los roles de un usuario) y POST /api/v1/productos/{id}/imagen (reemplaza la imagen del producto).

Definición de endpoints

Los 14 recursos exponen los mismos ocho endpoints; {recurso} se sustituye por la URI de la sección 3. Los siete procesos del CRUD quedan cubiertos, con la actualización dividida en total (PUT) y parcial (PATCH).

#	Proceso	Método	Endpoint	Efecto
1	Consultar listado	GET	/api/v1/{recurso}	Devuelve los registros activos, paginados
2	Consultar un registro	GET	/api/v1/{recurso}/{id}	Devuelve un registro por su identificador
3	Crear	POST	/api/v1/{recurso}	Inserta un registro nuevo
4a	Actualizar (total)	PUT	/api/v1/{recurso}/{id}	Reemplaza todos los campos editables
4b	Actualizar (parcial)	PATCH	/api/v1/{recurso}/{id}	Modifica solo los campos enviados
5	Eliminar lógicamente	DELETE	/api/v1/{recurso}/{id}	Marca deleted_at; el registro pasa a la papelera
6	Restaurar	PATCH	/api/v1/{recurso}/{id}/restaurar	Limpia deleted_at; el registro vuelve a estar activo
7	Eliminar físicamente	DELETE	/api/v1/{recurso}/{id}/permanente	Borra la fila de forma definitiva

Reglas de diseño:

- La papelera se consulta con el mismo listado: GET /api/v1/{recurso}?eliminados=solo.
- El borrado físico solo procede sobre registros ya eliminados lógicamente, igual que forceDestroy en el ProductoController actual.
- Restaurar usa PATCH porque modifica un solo atributo (deleted_at) de un recurso existente.
- GET, PUT y DELETE son idempotentes; POST no lo es.
- Las URI nombran recursos con sustantivos; la acción la expresa el método HTTP.

Métodos HTTP

La API usa cinco métodos HTTP; cada uno expresa una sola intención sobre el recurso.

Método	Uso en la API	Lleva cuerpo	Idempotente	Código de éxito
GET	Consultar listados y registros; usuario autenticado	No	Sí	200
POST	Crear registros; iniciar y cerrar sesión; subir imagen	Sí	No	201 (200 o 204 en autenticación)
PUT	Actualizar un registro completo; asignar roles	Sí	Sí	200
PATCH	Actualizar parcialmente; restaurar un registro	Sí (no al restaurar)	No	200
DELETE	Eliminar lógica o físicamente	No	Sí	204

Parámetros de entrada

Cada solicitud se define por la operación (tabla 6.2) y por el recurso (tabla 6.3). La URL completa es la URL base más el endpoint.

Encabezados

Header	Valor	Cuándo se envía
Accept	application/json	Siempre
Authorization	Bearer {token}	Siempre, excepto en POST /auth/login
Content-Type	application/json	En POST, PUT y PATCH con cuerpo
Content-Type	multipart/form-data	Solo al enviar la imagen de un producto

6.2 Solicitud por operación

Método y URL	Descripción del servicio	Parámetros de URL	Query parameters	Request body
GET /{recurso}	Lista paginada de registros	Ninguno	page, per_page, buscar, ordenar, eliminados y los filtros del recurso	No aplica
GET /{recurso}/{id}	Detalle de un registro	id (entero)	incluir (relaciones a anexar)	No aplica
POST /{recurso}	Crea un registro	Ninguno	Ninguno	Todos los campos obligatorios; opcionales si se desean
PUT /{recurso}/{id}	Actualización total	id (entero)	Ninguno	Todos los campos obligatorios
PATCH /{recurso}/{id}	Actualización parcial	id (entero)	Ninguno	Al menos un campo del recurso
DELETE /{recurso}/{id}	Eliminación lógica	id (entero)	Ninguno	No aplica
PATCH /{recurso}/{id} /restaurar	Restaura un registro eliminado	id (entero)	Ninguno	No aplica
DELETE /{recurso}/{id} /permanente	Eliminación física	id (entero)	Ninguno	No aplica

Query parameters comunes del listado:

Parámetro	Tipo	Valor por defecto	Uso
page	entero ≥ 1	1	Número de página
per_page	entero 1–100	15	Registros por página
buscar	texto	vacío	Búsqueda en los campos de texto del recurso
ordenar	campo o -campo	-id	Orden ascendente o descendente (prefijo -)
eliminados	no, con, solo	no	Excluye, incluye o muestra solo la papelera

6.3 Cuerpo y filtros por recurso

Recurso	Campos obligatorios	Campos opcionales	Filtros propios (query)
productos	nombre (texto 3–100), precio (decimal ≥ 0.01), stock (entero ≥ 0), categoria (texto)	imagen (jpeg, jpg, png o webp; máx. 2 MB; multipart)	categoria, precio_min, precio_max
usuarios	nombre (máx. 100), apellido (máx. 100), email (único, máx. 150), password (mín. 8; solo al crear)	telefono (máx. 20), activo (booleano)	activo, rol
roles	nombre (máx. 50)	descripcion (máx. 200)	Ninguno
especialidades	nombre (máx. 80)	descripcion	Ninguno
sesiones-sociales	id_usuario (existente), proveedor (máx. 50), provider_id (máx. 200)	Ninguno	id_usuario, proveedor
medicos	id_usuario (existente, único), cedula (única, máx. 30), especialidad_id (existente), consultorio (máx. 100)	Ninguno	especialidad_id

Recurso	Campos obligatorios	Campos opcionales	Filtros propios (query)
citas	paciente_id (existente), medico_id (existente), fecha (AAAA-MM-DD), hora (HH:MM)	estado (por defecto Pendiente), motivo	medico_id, paciente_id, estado, fecha_desde, fecha_hasta
expedientes	id_cita (existente, único), diagnostico, tratamiento	observaciones	id_cita
servicios-medicos	nombre (máx. 100), costo (decimal ≥ 0)	descripcion, activo (booleano)	activo
carritos	id_usuario (existente)	estado (por defecto Activo)	id_usuario, estado
carrito-detalles	id_carrito (existente), id_servicio (existente)	cantidad (entero ≥ 1 ; por defecto 1)	id_carrito
listas-deseos	id_usuario (existente), id_servicio (existente)	Ninguno	id_usuario
historial-transacciones	id_usuario (existente), id_carrito (existente), total (decimal ≥ 0), estado (máx. 30)	Ninguno	id_usuario, estado, fecha_desde, fecha_hasta
logs	id_usuario (existente), accion (máx. 100), modulo (máx. 100), ip (IP válida)	Ninguno	id_usuario, modulo, fecha_desde, fecha_hasta

Las fechas automáticas (fecha_registro, fecha_vinculacion, fecha, fecha_agregado, created_at) las asigna el servidor y no se envían.

Ejemplo: crear un producto

```
POST /api/v1/productos HTTP/1.1
Host: farmacia.test
Accept: application/json
Content-Type: application/json
Authorization: Bearer 3|kT9xQm...
```

```
{
  "nombre": "Paracetamol 500 mg",
  "precio": 35.50,
  "stock": 120,
  "categoria": "Analgésicos"
}
```

Servicios adicionales:

Método y URL	Descripción	Parámetros de URL	Request body
PUT /usuarios/{id}/roles	Sustituye los roles del usuario	id (entero)	{ "roles": [1, 3] } con ids de rol existentes
POST /productos/{id}/imagen	Reemplaza la imagen del producto	id (entero)	multipart/form-data con el archivo imagen

Respuestas esperadas

Todas las respuestas con cuerpo son JSON (Content-Type: application/json) y usan una de tres estructuras: registró, colección o error (sección 8).

Operación	Código de éxito	Estructura	Datos devueltos
Listado	200	Colección	data (arreglo de registros), meta (paginación), links (navegación)
Detalle	200	Registro	data con el registro completo y las relaciones pedidas en incluir
Crear	201	Registro	message, data con el registro creado; header Location con su URL
Actualizar (PUT o PATCH)	200	Registro	message, data con el registro ya actualizado
Eliminar lógicamente	204	Sin cuerpo	Ninguno
Restaurar	200	Registro	message, data con el registro restaurado (deleted_at en null)
Eliminar físicamente	204	Sin cuerpo	Ninguno

Registro

```
{
  "message": "Producto registrado correctamente.",
  "data": {
    "id": 12,
    "nombre": "Paracetamol 500 mg",
    "precio": 35.50,
    "stock": 120,
    "categoria": "Analgésicos",
    "imagen": "uploads/productos/Producto_12_1.webp",
    "created_at": "2026-10-04T16:20:00Z",
    "updated_at": "2026-10-04T16:20:00Z",
    "deleted_at": null
  }
}
```

Colección

```
{
  "data": [
    { "id": 12, "nombre": "Paracetamol 500 mg", "precio": 35.50, "stock": 120,
    "categoria": "Analgésicos" }
  ],
  "meta": { "current_page": 1, "per_page": 15, "total": 48, "last_page": 4 },
  "links": { "first": "/api/v1/productos?page=1", "prev": null, "next":
  "/api/v1/productos?page=2", "last": "/api/v1/productos?page=4" }
}
```

Datos devueltos por recurso

Recurso	Campos en data	Relaciones disponibles en incluir
productos	id, nombre, precio, stock, categoria, imagen, created_at, updated_at, deleted_at	Ninguna
usuarios	id_usuario, nombre, apellido, email, telefono, activo, fecha_registro, deleted_at	roles, medico
roles	id_rol, nombre, descripcion, deleted_at	usuarios
especialidades	id_especialidad, nombre, descripcion, deleted_at	medicos
sesiones-sociales	id_sesion, id_usuario, proveedor, fecha_vinculacion, deleted_at	usuario

Recurso	Campos en data	Relaciones disponibles en incluir
medicos	id_medico, id_usuario, cedula, especialidad_id, consultorio, deleted_at	usuario, especialidad
citas	id_cita, paciente_id, medico_id, fecha, hora, estado, motivo, deleted_at	paciente, medico, expediente
expedientes	id_expediente, id_cita, diagnostico, tratamiento, observaciones, fecha, deleted_at	cita
servicios-medicos	id_servicio, nombre, descripcion, costo, activo, deleted_at	Ninguna
carritos	id_carrito, id_usuario, fecha, estado, deleted_at	usuario, detalles
carrito-detalles	id_detalle, id_carrito, id_servicio, cantidad, deleted_at	carrito, servicio
listas-deseos	id_lista, id_usuario, id_servicio, fecha_agregado, deleted_at	usuario, servicio
historial-transacciones	id_transaccion, id_usuario, id_carrito, total, estado, fecha, deleted_at	usuario, carrito
logs	id_log, id_usuario, accion, modulo, ip, fecha, deleted_at	usuario

Por seguridad, `usuarios.password` y `sesiones_sociales.provider_id` nunca se devuelven. Las fechas van en ISO 8601 y los decimales como número.

Mensajes de error

Todo error devuelve la misma estructura JSON: `message` (texto para el usuario), `code` (clave para el cliente) y, solo en validaciones, `errors` por campo.

```
{
  "message": "Los datos enviados no son válidos.",
  "code": "VALIDACION_FALLIDA",
  "errors": {
    "nombre": ["El campo nombre es obligatorio."],
    "precio": ["El precio debe ser mayor o igual a 0.01."]
  }
}
```

Tipo de error	HTTP	code	Mensaje	Servicios donde ocurre
Datos inválidos (solicitud mal formada)	400	SOLICITUD_INVALIDA	"La solicitud no es válida: JSON mal formado o parámetro incorrecto."	Listado (query inválido), crear, actualizar, login
Datos inválidos (reglas de validación)	422	VALIDACION_FALLIDA	"Los datos enviados no son válidos." más el detalle por campo	Crear, actualizar, login, asignar roles, subir imagen
Campos obligatorios	422	VALIDACION_FALLIDA	"El campo {campo} es obligatorio."	Crear, PUT, login
Autenticación requerida	401	NO_AUTENTICADO	"Debes iniciar sesión para acceder a este recurso."	Todos, excepto login
Credenciales incorrectas	401	CREDENCIALES_INVALIDAS	"El correo o la contraseña son incorrectos."	Login
Acceso no autorizado	403	ACCESO_DENEGADO	"Tu cuenta no tiene acceso al área administrativa."	Login (cuenta inactiva o sin rol administrativo)
Permisos insuficientes	403	PERMISO_INSUFICIENTE	"No tienes el permiso {permiso}"	Todos los servicios protegidos

Tipo de error	HTTP	code	Mensaje	Servicios donde ocurre
			para realizar esta acción."	
Registro inexistente	404	NO_ENCONTRADO	"El {recurso} solicitado no existe."	Detalle, actualizar, eliminar, restaurar, borrado físico
Conflicto de integridad referencial	409	CONFLICTO_INTEGRIDAD	"No se puede eliminar el {recurso} porque está relacionado con: {dependencias}."	Eliminar y borrado físico
Valor duplicado	409	REGISTRO_DUPLICADO	"Ya existe un registro con ese {campo}."	Crear y actualizar (email, cédula, id_usuario de médico, id_cita de expediente)
Estado incompatible	409	ESTADO_INVALIDO	"El registro ya está activo." o "Solo se eliminan definitivamente registros en la papelera."	Restaurar y borrado físico
Error interno del servidor	500	ERROR_INTERNO	"Ocurrió un error inesperado. Intenta de nuevo más tarde."	Todos

El error 500 nunca expone trazas ni consultas SQL; el detalle se guarda en `storage/logs/laravel.log`.

Conflictos de integridad por recurso:

Recurso que se elimina	Bloqueado por
especialidades	médicos que la tienen asignada
medicos	citas del médico
servicios-medicos	detalles de carrito que lo incluyen
carritos	transacciones del historial
usuarios	Su borrado físico elimina en cascada médico, citas, carritos, listas, transacciones, logs y sesiones; se bloquea si esos registros dependen a su vez de una restricción anterior

9. Códigos HTTP

Los códigos dependen de la operación y aplican igual a los 14 recursos.

Endpoint	200	201	204	400	401	403	404	409	422	500
GET /{recurso}	Sí			Sí	Sí	Sí				Sí
GET /{recurso}/{id}	Sí				Sí	Sí	Sí			Sí
POST /{recurso}		Sí		Sí	Sí	Sí		Sí	Sí	Sí
PUT /{recurso}/{id}	Sí			Sí	Sí	Sí	Sí	Sí	Sí	Sí
PATCH /{recurso}/{id}	Sí			Sí	Sí	Sí	Sí	Sí	Sí	Sí
DELETE /{recurso}			Sí		Sí	Sí	Sí	Sí		Sí

Endpoint	200	201	204	400	401	403	404	409	422	500
urso}/{id}										
PATCH /{recurso}/{id}/restaurar	Sí				Sí	Sí	Sí	Sí		Sí
DELETE /{recurso}/{id}/permanente			Sí		Sí	Sí	Sí	Sí		Sí
PUT /usuarios/{id}/roles	Sí			Sí	Sí	Sí	Sí		Sí	Sí
POST /productos/{id}/imagen	Sí			Sí	Sí	Sí	Sí		Sí	Sí
POST /auth/login	Sí			Sí	Sí	Sí			Sí	Sí
GET /auth/me	Sí				Sí					Sí
POST /auth/logout			Sí		Sí					Sí

Código	Significado en esta API
200 OK	Consulta, actualización o restauración exitosa, con cuerpo
201 Created	Registro creado; incluye Location
204 No Content	Eliminación o cierre de sesión exitoso, sin cuerpo
400 Bad Request	JSON mal formado o parámetro de URL o query inválido
401 Unauthorized	Token ausente, vencido o revocado; credenciales incorrectas
403 Forbidden	Usuario autenticado sin el rol o permiso requerido
404 Not Found	El registro o la ruta no existe
409 Conflict	Integridad referencial, duplicado o estado incompatible
422 Unprocessable Entity	El cuerpo es JSON válido pero no cumple las reglas de validación
500 Internal Server Error	Falla no controlada del servidor

Servicios de autenticación

El área administrativa se autentica con tokens personales de Laravel Sanctum, enviados en `Authorization: Bearer {token}`. Sanctum ya está instalado (`config/sanctum.php` y la tabla `personal_access_tokens`).

Requisito de implementación: el modelo `Usuario` hoy extiende `Model`; debe extender `Authenticatable` y usar el trait `HasApiTokens` para emitir tokens.

Servicio	Método y URL	Headers	Request body	Respuesta de éxito	Errores
Inicio de sesión	POST <code>/api/v1/auth/login</code>	Accept, Content-Type	email (obligatorio), password (obligatorio)	200 con token, token_type, expires_at y usuario	400, 401 credenciales incorrectas, 403 cuenta inactiva o sin rol administrativo, 422, 500

Servicio	Método y URL	Headers	Request body	Respuesta de éxito	Errores
Usuario autenticado	GET /api/v1/auth/me	Accept, Authorization	No aplica	200 con data: datos del usuario, roles y permisos	401, 500
Cierre de sesión	POST /api/v1/auth/logout	Accept, Authorization	No aplica	204 sin cuerpo; el token actual queda revocado	401, 500

Solicitud y respuesta del inicio de sesión:

```
{ "email": "admin@farmaciavital.mx", "password": "*****" }
```

```
{
  "message": "Inicio de sesión correcto.",
  "data": {
    "token": "3|kT9xQm...",
    "token_type": "Bearer",
    "expires_at": "2026-10-05T00:20:00Z",
    "usuario": {
      "id_usuario": 1,
      "nombre": "Héctor",
      "apellido": "Hernández",
      "email": "admin@farmaciavital.mx",
      "roles": ["Administrador"]
    }
  }
}
```

Reglas del proceso:

- Login verifica el password contra el hash, que activo sea verdadero y que el usuario tenga al menos un rol administrativo.
- El token dura 8 horas; al vencer, cualquier servicio responde 401 y el cliente debe iniciar sesión de nuevo.
- Login se limita a 5 intentos por minuto por IP; al excederlo responde 429 Too Many Requests.
- Cada inicio y cierre de sesión se registra en la tabla logs (acción, módulo, IP).

Servicios de autorización

Todos los endpoints requieren autenticación, excepto POST /auth/login. La protección tiene dos capas de middleware: auth:sanctum valida el token (401 si falla) y permiso:{recurso}.{accion} valida el permiso del rol (403 si falla).

Cada recurso tiene seis permisos con el formato {recurso}.{accion}:

Acción	Permiso	Endpoints que protege
Ver	{recurso}.ver	GET listado y GET detalle
Crear	{recurso}.crear	POST
Editar	{recurso}.editar	PUT y PATCH
Eliminar	{recurso}.eliminar	DELETE lógico
Restaurar	{recurso}.restaurar	PATCH restaurar y listado con eliminados=solo o con
Destruir	{recurso}.destruir	DELETE permanente

Roles autorizados por módulo:

Módulo	Administrador	Farmacéutico	Médico
productos	Todos	ver, crear, editar, eliminar, restaurar	ver
servicios-medicos	Todos	ver, crear, editar, eliminar, restaurar	ver
carritos	Todos	ver, editar	Sin acceso
carrito-detalles	Todos	ver, editar	Sin acceso
listas-deseos	Todos	ver	Sin acceso
historial-transacciones	Todos	ver	Sin acceso
especialidades	Todos	ver	ver
medicos	Todos	ver	ver
citas	Todos	ver	ver, crear, editar, eliminar, restaurar
expedientes	Todos	Sin acceso	ver, crear, editar
usuarios	Todos	Sin acceso	ver
roles	Todos	Sin acceso	Sin acceso
sesiones-sociales	Todos	Sin acceso	Sin acceso
logs	Todos	Sin acceso	Sin acceso

Reglas:

- El permiso destruir y el servicio PUT /usuarios/{id}/roles (permiso usuarios.asignar-roles) son exclusivos del Administrador.
- El rol Paciente no tiene permisos administrativos: su login en esta API responde 403.
- El Médico solo ve y modifica las citas y expedientes donde él es el médico asignado; otro registro responde 403.
- La tabla roles no tiene permisos asociados. El mapa rol → permisos se define en config/permisos.php; migrarlo a tablas permisos y rol_permisos queda como mejora.
- GET /auth/me y POST /auth/logout requieren autenticación pero ningún permiso.

Tabla resumen de la API

La API tiene 117 servicios: 3 de autenticación, 112 de CRUD (8 por cada uno de los 14 recursos) y 2 adicionales. Todas las rutas llevan el prefijo /api/v1.

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
Autenticación	/auth/login	POST	Inicia sesión y emite el token	No	Ninguno	200, 400, 401, 403, 422, 500
Autenticación	/auth/me	GET	Devuelve el usuario autenticado	Sí	Ninguno	200, 401, 500
Autenticación	/auth/logout	POST	Cierra sesión y revoca el token	Sí	Ninguno	204, 401, 500
Productos	/productos	GET	Lista productos	Sí	productos.ver	200, 400, 401, 403, 500
Productos	/productos/{id}	GET	Consulta un producto	Sí	productos.ver	200, 401, 403, 404, 500
Productos	/productos	POST	Crea un producto	Sí	productos.crear	201, 400, 401, 403, 409, 422, 500

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
Productos	/productos/{id}	PUT	Actualiza un producto completo	Sí	productos.editar	200, 400, 401, 403, 404, 409, 422, 500
Productos	/productos/{id}	PATCH	Actualiza un producto parcialmente	Sí	productos.editar	200, 400, 401, 403, 404, 409, 422, 500
Productos	/productos/{id}	DELETE	Elimina lógicamente un producto	Sí	productos.eliminar	204, 401, 403, 404, 409, 500
Productos	/productos/{id}/restaurar	PATCH	Restaura un producto eliminado	Sí	productos.restaurar	200, 401, 403, 404, 409, 500
Productos	/productos/{id}/permanente	DELETE	Elimina físicamente un producto	Sí	productos.destruir	204, 401, 403, 404, 409, 500
Productos	/productos/{id}/imagen	POST	Reemplaza la imagen del producto	Sí	productos.editar	200, 400, 401, 403, 404, 422, 500
Usuarios	/usuarios	GET	Lista usuarios	Sí	usuarios.ver	200, 400, 401, 403, 500
Usuarios	/usuarios/{id}	GET	Consulta un usuario	Sí	usuarios.ver	200, 401, 403, 404, 500
Usuarios	/usuarios	POST	Crea un usuario	Sí	usuarios.crear	201, 400, 401, 403, 409, 422, 500
Usuarios	/usuarios/{id}	PUT	Actualiza un usuario completo	Sí	usuarios.editar	200, 400, 401, 403, 404, 409, 422, 500

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
Usuarios	/usuarios/{id}	PATCH	Actualiza un usuario parcialmente	Sí	usuarios.editar	200, 400, 401, 403, 404, 409, 422, 500
Usuarios	/usuarios/{id}	DELETE	Elimina lógicamente un usuario	Sí	usuarios.eliminar	204, 401, 403, 404, 409, 500
Usuarios	/usuarios/{id}/restaurar	PATCH	Restaura un usuario eliminado	Sí	usuarios.restaurar	200, 401, 403, 404, 409, 500
Usuarios	/usuarios/{id}/permanente	DELETE	Elimina físicamente un usuario	Sí	usuarios.destruir	204, 401, 403, 404, 409, 500
Usuarios	/usuarios/{id}/roles	PUT	Asigna los roles del usuario	Sí	usuarios.asignar-roles	200, 400, 401, 403, 404, 422, 500
Roles	/roles	GET	Lista roles	Sí	roles.ver	200, 400, 401, 403, 500
Roles	/roles/{id}	GET	Consulta un rol	Sí	roles.ver	200, 401, 403, 404, 500
Roles	/roles	POST	Crea un rol	Sí	roles.crear	201, 400, 401, 403, 409, 422, 500
Roles	/roles/{id}	PUT	Actualiza un rol completo	Sí	roles.editar	200, 400, 401, 403, 404, 409, 422, 500
Roles	/roles/{id}	PATCH	Actualiza un rol parcialmente	Sí	roles.editar	200, 400, 401, 403, 404, 409, 422, 500

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
Roles	/roles/{id}	DELETE	Elimina lógicamente un rol	Sí	roles.eliminar	204, 401, 403, 404, 409, 500
Roles	/roles/{id}/restaurar	PATCH	Restaura un rol eliminado	Sí	roles.restaurar	200, 401, 403, 404, 409, 500
Roles	/roles/{id}/permanente	DELETE	Elimina físicamente un rol	Sí	roles.destruir	204, 401, 403, 404, 409, 500
Especialidades	/especialidades	GET	Lista especialidades	Sí	especialidades.ver	200, 400, 401, 403, 500
Especialidades	/especialidades/{id}	GET	Consulta una especialidad	Sí	especialidades.ver	200, 401, 403, 404, 500
Especialidades	/especialidades	POST	Crea una especialidad	Sí	especialidades.crear	201, 400, 401, 403, 409, 422, 500
Especialidades	/especialidades/{id}	PUT	Actualiza una especialidad completa	Sí	especialidades.editar	200, 400, 401, 403, 404, 409, 422, 500
Especialidades	/especialidades/{id}	PATCH	Actualiza una especialidad parcialmente	Sí	especialidades.editar	200, 400, 401, 403, 404, 409, 422, 500
Especialidades	/especialidades/{id}	DELETE	Elimina lógicamente una especialidad	Sí	especialidades.eliminar	204, 401, 403, 404, 409, 500
Especialidades	/especialidades/{id}/restaurar	PATCH	Restaura una	Sí	especialidades.restaurar	200, 401, 403, 404, 409, 500

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
			especialidad eliminada			
Especialidades	/especialidades/{id}/permanente	DELETE	Elimina físicamente una especialidad	Sí	especialidades.destruir	204, 401, 403, 404, 409, 500
Sesiones sociales	/sesiones-sociales	GET	Lista sesiones sociales	Sí	sesiones-sociales.ver	200, 400, 401, 403, 500
Sesiones sociales	/sesiones-sociales/{id}	GET	Consulta una sesión social	Sí	sesiones-sociales.ver	200, 401, 403, 404, 500
Sesiones sociales	/sesiones-sociales	POST	Crea una sesión social	Sí	sesiones-sociales.crear	201, 400, 401, 403, 409, 422, 500
Sesiones sociales	/sesiones-sociales/{id}	PUT	Actualiza una sesión social completa	Sí	sesiones-sociales.editar	200, 400, 401, 403, 404, 409, 422, 500
Sesiones sociales	/sesiones-sociales/{id}	PATCH	Actualiza una sesión social parcialmente	Sí	sesiones-sociales.editar	200, 400, 401, 403, 404, 409, 422, 500
Sesiones sociales	/sesiones-sociales/{id}	DELETE	Elimina lógicamente una sesión social	Sí	sesiones-sociales.eliminar	204, 401, 403, 404, 409, 500
Sesiones sociales	/sesiones-sociales/{id}/restaurar	PATCH	Restaura una sesión social eliminada	Sí	sesiones-sociales.restaurar	200, 401, 403, 404, 409, 500

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
Sesiones sociales	/sesiones-sociales/{id}/permanente	DELETE	Elimina físicamente una sesión social	Sí	sesiones-sociales.destruir	204, 401, 403, 404, 409, 500
Médicos	/medicos	GET	Lista médicos	Sí	medicos.ver	200, 400, 401, 403, 500
Médicos	/medicos/{id}	GET	Consulta un médico	Sí	medicos.ver	200, 401, 403, 404, 500
Médicos	/medicos	POST	Crea un médico	Sí	medicos.crear	201, 400, 401, 403, 409, 422, 500
Médicos	/medicos/{id}	PUT	Actualiza un médico completo	Sí	medicos.editar	200, 400, 401, 403, 404, 409, 422, 500
Médicos	/medicos/{id}	PATCH	Actualiza un médico parcialmente	Sí	medicos.editar	200, 400, 401, 403, 404, 409, 422, 500
Médicos	/medicos/{id}	DELETE	Elimina lógicamente un médico	Sí	medicos.eliminar	204, 401, 403, 404, 409, 500
Médicos	/medicos/{id}/restaurar	PATCH	Restaura un médico eliminado	Sí	medicos.restaurar	200, 401, 403, 404, 409, 500
Médicos	/medicos/{id}/permanente	DELETE	Elimina físicamente un médico	Sí	medicos.destruir	204, 401, 403, 404, 409, 500
Citas	/citas	GET	Lista citas	Sí	citas.ver	200, 400, 401, 403, 500

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
Citas	/citas/{id}	GET	Consulta una cita	Sí	citas.ver	200, 401, 403, 404, 500
Citas	/citas	POST	Crea una cita	Sí	citas.crear	201, 400, 401, 403, 409, 422, 500
Citas	/citas/{id}	PUT	Actualiza una cita completa	Sí	citas.editar	200, 400, 401, 403, 404, 409, 422, 500
Citas	/citas/{id}	PATCH	Actualiza una cita parcialmente	Sí	citas.editar	200, 400, 401, 403, 404, 409, 422, 500
Citas	/citas/{id}	DELETE	Elimina lógicamente una cita	Sí	citas.eliminar	204, 401, 403, 404, 409, 500
Citas	/citas/{id}/restaurar	PATCH	Restaura una cita eliminada	Sí	citas.restaurar	200, 401, 403, 404, 409, 500
Citas	/citas/{id}/permanente	DELETE	Elimina físicamente una cita	Sí	citas.destruir	204, 401, 403, 404, 409, 500
Expedientes	/expedientes	GET	Lista expedientes	Sí	expedientes.ver	200, 400, 401, 403, 500
Expedientes	/expedientes/{id}	GET	Consulta un expediente	Sí	expedientes.ver	200, 401, 403, 404, 500
Expedientes	/expedientes	POST	Crea un expediente	Sí	expedientes.crear	201, 400, 401, 403, 409, 422, 500

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
Expedientes	/expedientes/{id}	PUT	Actualiza un expediente completo	Sí	expedientes.editar	200, 400, 401, 403, 404, 409, 422, 500
Expedientes	/expedientes/{id}	PATCH	Actualiza un expediente parcialmente	Sí	expedientes.editar	200, 400, 401, 403, 404, 409, 422, 500
Expedientes	/expedientes/{id}	DELETE	Elimina lógicamente un expediente	Sí	expedientes.eliminar	204, 401, 403, 404, 409, 500
Expedientes	/expedientes/{id}/restaurar	PATCH	Restaura un expediente eliminado	Sí	expedientes.restaurar	200, 401, 403, 404, 409, 500
Expedientes	/expedientes/{id}/permanente	DELETE	Elimina físicamente un expediente	Sí	expedientes.destruir	204, 401, 403, 404, 409, 500
Servicios médicos	/servicios-medicos	GET	Lista servicios médicos	Sí	servicios-medicos.ver	200, 400, 401, 403, 500
Servicios médicos	/servicios-medicos/{id}	GET	Consulta un servicio médico	Sí	servicios-medicos.ver	200, 401, 403, 404, 500
Servicios médicos	/servicios-medicos	POST	Crea un servicio médico	Sí	servicios-medicos.crear	201, 400, 401, 403, 409, 422, 500
Servicios médicos	/servicios-medicos/{id}	PUT	Actualiza un servicio médico completo	Sí	servicios-medicos.editar	200, 400, 401, 403, 404, 409, 422, 500

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
Servicios médicos	/servicios-medicos/{id}	PATCH	Actualiza un servicio médico parcialmente	Sí	servicios-medicos.editar	200, 400, 401, 403, 404, 409, 422, 500
Servicios médicos	/servicios-medicos/{id}	DELETE	Elimina lógicamente un servicio médico	Sí	servicios-medicos.eliminar	204, 401, 403, 404, 409, 500
Servicios médicos	/servicios-medicos/{id}/restaurar	PATCH	Restaura un servicio médico eliminado	Sí	servicios-medicos.restaurar	200, 401, 403, 404, 409, 500
Servicios médicos	/servicios-medicos/{id}/permanente	DELETE	Elimina físicamente un servicio médico	Sí	servicios-medicos.destruir	204, 401, 403, 404, 409, 500
Carritos	/carritos	GET	Lista carritos	Sí	carritos.ver	200, 400, 401, 403, 500
Carritos	/carritos/{id}	GET	Consulta un carrito	Sí	carritos.ver	200, 401, 403, 404, 500
Carritos	/carritos	POST	Crea un carrito	Sí	carritos.crear	201, 400, 401, 403, 409, 422, 500
Carritos	/carritos/{id}	PUT	Actualiza un carrito completo	Sí	carritos.editar	200, 400, 401, 403, 404, 409, 422, 500
Carritos	/carritos/{id}	PATCH	Actualiza un carrito parcialmente	Sí	carritos.editar	200, 400, 401, 403, 404, 409, 422, 500

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
Carritos	/carritos/{id}	DELETE	Elimina lógicamente un carrito	Sí	carritos.eliminar	204, 401, 403, 404, 409, 500
Carritos	/carritos/{id}/restaurar	PATCH	Restaura un carrito eliminado	Sí	carritos.restaurar	200, 401, 403, 404, 409, 500
Carritos	/carritos/{id}/permanente	DELETE	Elimina físicamente un carrito	Sí	carritos.destruir	204, 401, 403, 404, 409, 500
Detalles de carrito	/carrito-detalles	GET	Lista detalles de carrito	Sí	carrito-detalles.ver	200, 400, 401, 403, 500
Detalles de carrito	/carrito-detalles/{id}	GET	Consulta un detalle de carrito	Sí	carrito-detalles.ver	200, 401, 403, 404, 500
Detalles de carrito	/carrito-detalles	POST	Crea un detalle de carrito	Sí	carrito-detalles.crear	201, 400, 401, 403, 409, 422, 500
Detalles de carrito	/carrito-detalles/{id}	PUT	Actualiza un detalle completo	Sí	carrito-detalles.editar	200, 400, 401, 403, 404, 409, 422, 500
Detalles de carrito	/carrito-detalles/{id}	PATCH	Actualiza un detalle parcialmente	Sí	carrito-detalles.editar	200, 400, 401, 403, 404, 409, 422, 500
Detalles de carrito	/carrito-detalles/{id}	DELETE	Elimina lógicamente un detalle	Sí	carrito-detalles.eliminar	204, 401, 403, 404, 409, 500
Detalles de carrito	/carrito-detalles/{id}/restaurar	PATCH	Restaura un detalle eliminado	Sí	carrito-detalles.restaurar	200, 401, 403, 404, 409, 500

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
Detalles de carrito	/carrito-detalles/{id}/permanente	DELETE	Elimina físicamente un detalle	Sí	carrito-detalles.destruir	204, 401, 403, 404, 409, 500
Listas de deseos	/listas-de-seos	GET	Lista elementos de listas de deseos	Sí	listas-deseos.ver	200, 400, 401, 403, 500
Listas de deseos	/listas-de-seos/{id}	GET	Consulta un elemento de lista de deseos	Sí	listas-deseos.ver	200, 401, 403, 404, 500
Listas de deseos	/listas-de-seos	POST	Agrega un elemento a una lista de deseos	Sí	listas-deseos.crear	201, 400, 401, 403, 409, 422, 500
Listas de deseos	/listas-de-seos/{id}	PUT	Actualiza un elemento completo	Sí	listas-deseos.editar	200, 400, 401, 403, 404, 409, 422, 500
Listas de deseos	/listas-de-seos/{id}	PATCH	Actualiza un elemento parcialmente	Sí	listas-deseos.editar	200, 400, 401, 403, 404, 409, 422, 500
Listas de deseos	/listas-de-seos/{id}	DELETE	Elimina lógicamente un elemento	Sí	listas-deseos.eliminar	204, 401, 403, 404, 409, 500
Listas de deseos	/listas-de-seos/{id}/restaurar	PATCH	Restaura un elemento eliminado	Sí	listas-deseos.restaurar	200, 401, 403, 404, 409, 500
Listas de deseos	/listas-de-seos/{id}/permanente	DELETE	Elimina físicamente un elemento	Sí	listas-deseos.destruir	204, 401, 403, 404, 409, 500

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
Historial de transacciones	/historial-transacciones	GET	Lista transacciones	Sí	historial-transacciones.ver	200, 400, 401, 403, 500
Historial de transacciones	/historial-transacciones/{id}	GET	Consulta una transacción	Sí	historial-transacciones.ver	200, 401, 403, 404, 500
Historial de transacciones	/historial-transacciones	POST	Registra una transacción	Sí	historial-transacciones.crear	201, 400, 401, 403, 409, 422, 500
Historial de transacciones	/historial-transacciones/{id}	PUT	Actualiza una transacción completa	Sí	historial-transacciones.editar	200, 400, 401, 403, 404, 409, 422, 500
Historial de transacciones	/historial-transacciones/{id}	PATCH	Actualiza una transacción parcialmente	Sí	historial-transacciones.editar	200, 400, 401, 403, 404, 409, 422, 500
Historial de transacciones	/historial-transacciones/{id}	DELETE	Elimina lógicamente una transacción	Sí	historial-transacciones.eliminar	204, 401, 403, 404, 409, 500
Historial de transacciones	/historial-transacciones/{id}/restaurar	PATCH	Restaura una transacción eliminada	Sí	historial-transacciones.restaurar	200, 401, 403, 404, 409, 500
Historial de transacciones	/historial-transacciones/{id}/permanente	DELETE	Elimina físicamente una transacción	Sí	historial-transacciones.destruir	204, 401, 403, 404, 409, 500
Logs	/logs	GET	Lista logs de auditoría	Sí	logs.ver	200, 400, 401, 403, 500

Módulo	Endpoint	Método HTTP	Descripción	Requiere autenticación	Permiso requerido	Códigos de respuesta
Logs	/logs/{id}	GET	Consulta un log	Sí	logs.ver	200, 401, 403, 404, 500
Logs	/logs	POST	Registra un log	Sí	logs.crear	201, 400, 401, 403, 409, 422, 500
Logs	/logs/{id}	PUT	Actualiza un log completo	Sí	logs.editar	200, 400, 401, 403, 404, 409, 422, 500
Logs	/logs/{id}	PATCH	Actualiza un log parcialmente	Sí	logs.editar	200, 400, 401, 403, 404, 409, 422, 500
Logs	/logs/{id}	DELETE	Elimina lógicamente un log	Sí	logs.eliminar	204, 401, 403, 404, 409, 500
Logs	/logs/{id}/restaurar	PATCH	Restaura un log eliminado	Sí	logs.restaurar	200, 401, 403, 404, 409, 500
Logs	/logs/{id}/permanente	DELETE	Elimina físicamente un log	Sí	logs.destruir	204, 401, 403, 404, 409, 500

Los roles autorizados para cada permiso están en la matriz de la sección 11.

Conclusiones

El diseño convierte las 14 entidades del proyecto en una API REST uniforme: los mismos ocho endpoints, la misma estructura de respuesta y los mismos códigos de estado en todos los módulos.

- **Uniformidad.** Quien aprende a consumir un recurso sabe consumir los demás, lo que simplifica el desarrollo de clientes web, móviles o de punto de venta.
- **Seguridad de los datos.** La eliminación lógica con restauración evita pérdidas accidentales, y el borrado físico queda restringido y protegido contra conflictos de integridad referencial.
- **Control de acceso.** La autenticación con token y la autorización por permisos garantizan que cada rol solo opere los módulos que le corresponden.
- **Comunicación clara.** Los códigos HTTP y los mensajes de error estandarizados permiten al cliente saber qué falló y cómo corregirlo.

El diseño confirma la conclusión de la actividad anterior: REST es adecuado para Farmacia por su ligereza y su facilidad de integración. Para implementarlo faltan tres pasos: agregar `softDeletes()`, habilitar tokens en el modelo `Usuario` y crear los controladores de la API a partir de la lógica ya probada en productos.

Enlaces.

Video: <https://drive.google.com/file/d/12m43aCGYzXTG7tXRnNXn8V3HwGJJNsLt/view?usp=sharing>

Github: <https://github.com/Andyor-prog/Farmacia>